Institut Supérieur d'Informatique et Mathématiques
Monastir

Bases de données NOSQL

Chapitre 3: Introduction aux bases de données NOSQL

Dernière Mise à jour:
Janvier 2023

Asma KERKENI asma.kerkeni@gmail.com

Références

- P. Lacomme, S. Aridhi, R. Phan, Bases de données NoSQL et Big Data: Concevoir des bases de données pour le Big Data, Cours et travaux pratiques Ellipses ~ Technosup, 2/12/2014, 336 p., 9782340002616.
- Rudi Bruchez, L. Les bases de données NoSQL et le Big Data: Comprendre et mettre en œuvre, Ed. Eyrolles, 2015.
- Cours Bases de données NOSQL de Raja CHIKY, Institut Mines Télécom: http://perso.isep.fr/rchiky/
- Cours Big Data et NoSQL de Lilia Sfaxi, INSAT, Tunisie: http://liliasfaxi.wixsite.com/liliasfaxi
- https://db-engines.com/en/ranking
- http://b3d.bdpedia.fr/
- https://chewbii.com/

Objectifs

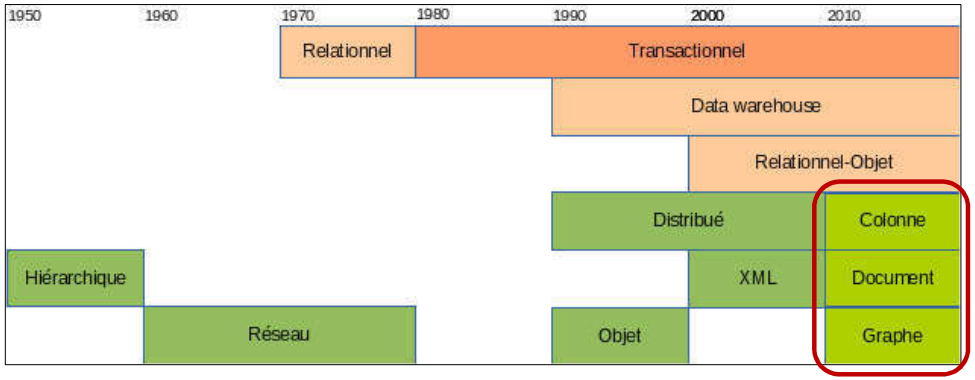
- Au terme de ce chapitre, vous serez capable de:
  - Citer les différents types de bases de données.
  - Décrire le contexte d'apparition de la mouvance NOSQL.
  - Définir les propriétés ACID et BASE et le théorème CAP
  - Comparer les bases de données relationnelles et les bases NOSQL.
  - Enumérer et caractériser les bases de données NOSQL.

Plan

- Historique des bases de données
- Définition du NOSQL
- Big Data et NOSQL
- Fondamentaux des modèles NoSQL
- SQL vs NOSQL
- Théorème de CAP
- Types de bases de données NOSQL

Vision Historique des bases de données

5



- Les BD NoSQL remettent en cause l'hégémonie des SGBDR telle qu'elle s'est constituée dans les années 1980.
- Les BD NoSQL sont essentiellement un retour à des modèles de données antérieurs (hiérarchique, réseau,...) qui existaient dans les années 60.

Vision Historique des bases de données:

-Dominance du relationnel-

6

- Le modèle relationnel assure le contrôle de l'intégrité des données, grâce à un modèle théorique puissant et simple, basé sur:
  - Le schéma : on peut exprimer des règles de cohérence a priori et déléguer leur contrôle au système.
  - La normalisation : on peut contrôler des contraintes (*via* le contrôle de la redondance) par un mécanisme de décomposition et retrouver l'information consolidée par les jointures.
  - La transaction : le système assure le maintien d'états cohérents au sein d'environnements concurrents et susceptibles de pannes.
- BD transactionnelle*:* Une base données classique est destinée à assumer des **transactions** en temps réel :
  - Ajout, mise à jour suppression de données
  - Questions sur des données identifiées ou questions statistiques

NOSQL: C'est quoi?

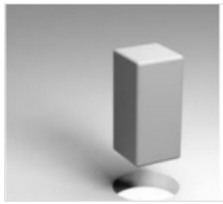
11

- Nom exact: Bases de données non relationnelles.
- Privilégier NOSQL plutôt que NoSQL.
- Ce n'est pas du relationnel, et le contexte d'utilisation n'est donc pas celui des SGBDR.
- Ce n'est pas seulement l'opposition à SQL :
  - il s'agit de compléments aux SGBDR pour des besoins spécifiques et non de solutions de remplacement.
  - expérimentations, autres modèles de données très simples,
  - nouveaux besoins, nouveaux outils !
  - logiciels de stockage de données plutôt que SGBD



NOSQL: C'est quoi?

12



"The whole point of seeking alternatives [to RDBMS systems] is that you need to solve a problem that relational databases are a bad fit for."

Eric Evans
Rackspace



- Un couple de concepteurs de bases de données entre dans un restaurant NoSQL. Une heure après, ils ressortent sans avoir réussi à manger.
  - Pourquoi?

Emergence du NOSQL: Big Data

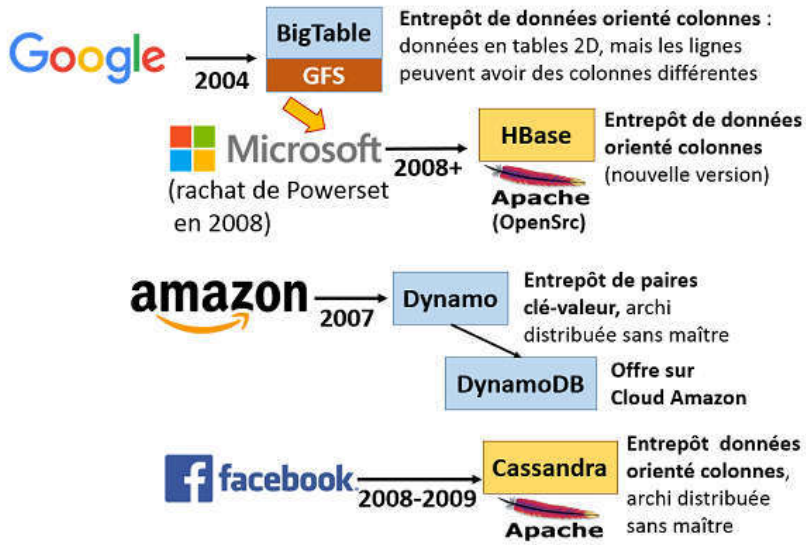
13

- **Origine :** recherche d'information sur le Web, moteurs type Google, Yahoo, données des réseaux sociaux,...
  - Besoin de stockage **d'énormes masses de données**.
  - Twitter par exemple reçoit plusieurs Tera-octets de données par jour.
  - Système distribué
  - table d'associations - Map - de couples (clef, valeur)
  - Différentes approches, rangées dans la famille "NOSQL".
- Ces technologies sont passées des acteurs du web au « grand public »

  →NOSQL n'est qu'une partie de cette vaste problématique du Big Data..

NOSQL: Partie du Big Data

14



Fondamentaux des modèles NoSQL

16

- Logique de dépôt uniquement (stocker et retrouver):
  - c'est la couche applicative qui fait tout le travail (traitement, cohérence...).
- Modèle de stockage clé-valeur:
  - Les données sont des valeurs auxquelles on accède par une clé artificielle.
  - Les données ne sont plus des lignes stockées dans des tables.
- Identification: Les clés d'identification sont des clés artificielles, en général unique à l'échelle de la BD, voire du monde :
  - Object Identifiers (OID)
  - Universally Unique IDentifier (UUID)
  - Uniform Resource Identifier (URI)

Fondamentaux des modèles NoSQL

17

- Imbrication: Structure des valeurs stockées connue par le serveur (JSON, XML, structure interne de type colonne...)
- Partitionnement et distribution (sharding): Les données sont partitionnées horizontalement et distribuées sur les nœuds d'un cluster. On utilise par exemple une clé de hashage pour distribuer les données.
- Schema-less: Les bases NOSQL se fondent sur une approche dite *schema-less*, c'est à dire sans schéma logique défini à priori.
  - Souplesse et rapidité, mais moins de contrôle et donc de cohérence des données.
  - Le mouvement NOSQL tend à réintégrer des fonctions de schématisation à priori (comme pour XML): le schéma est optionnel, mais conseillé en contexte de contrôle de cohérence.

Modèle de stockage clé-valeur

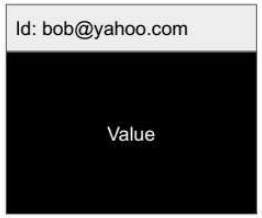
Trade-offs

Relational Databases

Id	bob@yahoo.com
Name	Bob Smith
Status	single
Avatar	bob.jpg
...	...

All data exposed
Powerful queries
Slow

Key Value Stores



Value is obscured
Only **get(key)**
Lightning fast
Easier to Distribute

NOSQL: fin du relationnel?

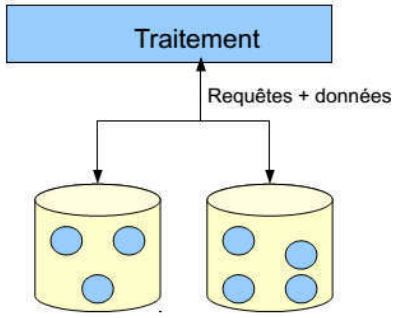
- BDR ne doit pas mourir mais des solutions de stockage doivent être envisagées pour des applications particulières (applications web en particulier)
  - Besoins des BDRs: schémas, cohérence forte, transactions,
  - Avantages des BDRs : matures et opérationnelles, expertise disponible, puissantes et stables.
  - ➢ Pas près de disparaitre à court et moyen terme
- ...mais elles ne sont plus suffisantes
  - Lourdeur inutile pour le stockage de certaines données
  - Systèmes hybrides combinant plusieurs technologies : de manière concurrente, coopérative, répartie, redondante

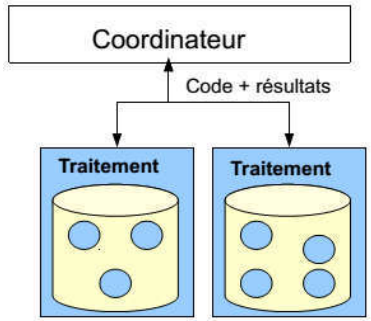
Quand utiliser NOSQL?

- Grand volume de données:
  - Besoin d'utiliser une architecture fortement distribuée
  - Google, Amazon, Yahoo, Facebook : 10-100K serveurs
- Requêtes massives:
  - Eviter les jointures car très gourmandes en temps de traitement
- Schéma évolutif:
  - Flexibilité et évolutivité du schéma à grande échelle
- Pas besoin de : transaction/ forte cohérence/intégrité/ requêtes complexes

Relationnel vs NOSQL

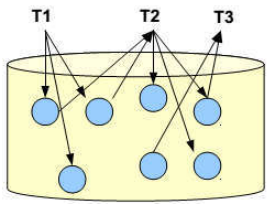
Relationnel	NOSQL
Traitements centralisés	Traitements distribués
Accès distribué	Accès local

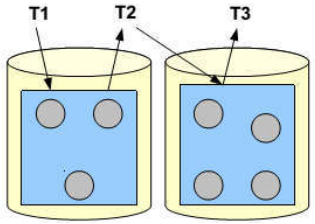




Relationnel vs NOSQL

Relationnel	NOSQL
☐ Accès à grain fin:	☐ Accès « batch » :
☐ beaucoup de lectures/écritures de petits objets.	☐ peu de lectures/ écritures de grands objets





Relationnel vs NOSQL (ACID vs Base)

Propriétés ACID pour les transactions

- ☐ **Atomicité** : une transaction s'effectue entièrement ou pas du tout
- ☐ **Cohérence** : le contenu d'une base doit être cohérent au début et à la fin d'une transaction.
- ☐ **Isolation** : les modifications d'une transaction ne sont visibles/modifiables que quand celle-ci a validé.
- ☐ **Durabilité :** une fois la transaction validée, l'état de la base est permanent (non affecté par les pannes ou autre.)

Systèmes distribués : modèle BASE

- ☐ **Basically Available** : garantie minimale pour taux de disponibilité face au grande quantité de requêtes.
- ☐ **Soft-state :** l'état du système peut changer au cours du temps même sans nouveaux inputs (cela est dû au modèle de consistance).
- ☐ **Eventually consistent** : tous les réplicas atteignent le même état, et le système devient à un moment consistant, si on stoppe les inputs.

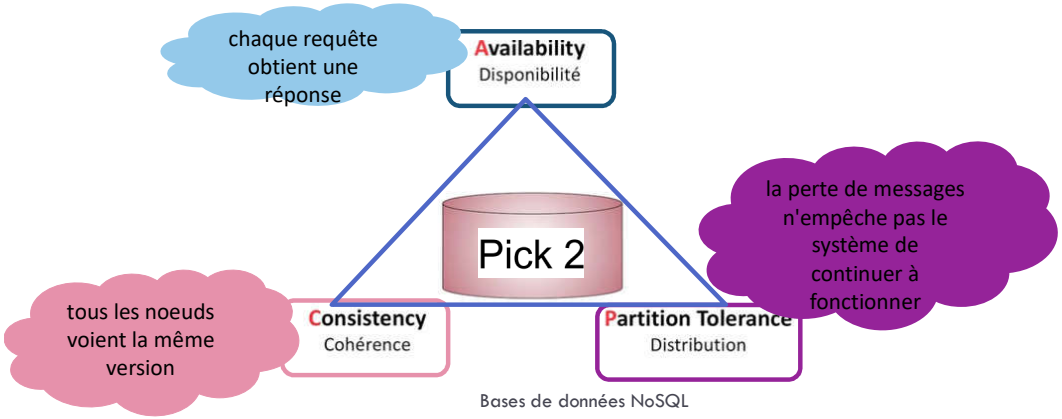


ACID ou BASE

- ☐ Le modèle ACID est parfois trop contraignant :
  - Système de réservation de billets d'avion? Modèle ACID nécessaire.
  - Calcul du nombre de messages envoyé par utilisateurs de chaque tranche d'âge, sur Facebook ? Modèle ACID inutile.

Théorème de CAP

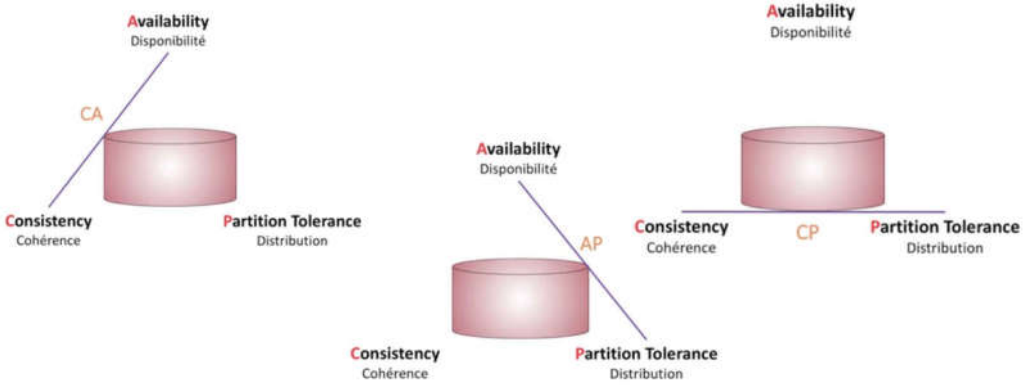
- ☐ En 2000, Eric A. Brewer a formalisé un théorème très intéressant reposant sur 3 propriétés fondamentales pour caractériser les bases de données (relationnelles, NoSQL et autres) :



Théorème de CAP

Au maximum **2 propriétés** respectées par n'importe quel système de données.

- plus précisément, si une garantie haute est assurée pour deux des propriétés, une garantie plus faible est assurée pour la dernière.

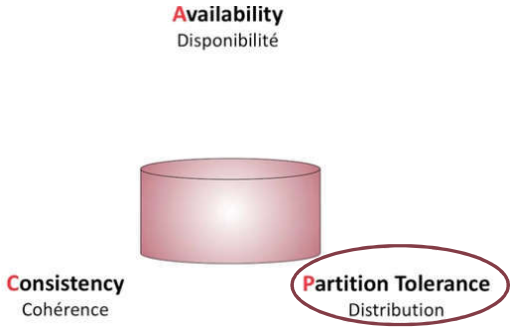


Théorème de CAP

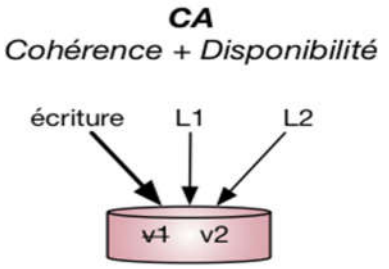
Au maximum **2 propriétés** respectées par n'importe quel système de données distribué:

- plus précisément, si une garantie haute est assurée pour deux des propriétés, une garantie plus faible est assurée pour la dernière.

- Pour passer à l'échelle, on utilise le **partitionnement.** Ce qui implique un **choix entre la cohérence ou la disponibilité.**

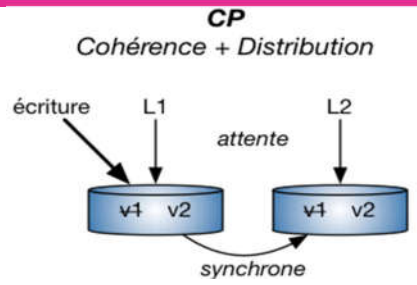


Théorème de CAP



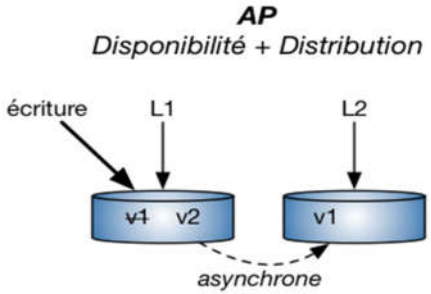
Lors d'opérations concurrentes sur une même donnée, les requêtes L1 et L2 retournent la nouvelle version (v2) et sans délai d'attente. Cette combinaison n'est possible que dans le cadre de bases de données transactionnelles telles que les SGBDR.

Théorème de CAP



En même temps, il est nécessaire de vérifier la cohérence des données en garantissant la valeur retournée malgré des mises à jour concurrentielles. La gestion de cette cohérence nécessite un protocole de synchronisation des réplicas, introduisant des délais de latence dans les temps de réponse (L1 et L2 attendent la synchronisation pour voir v2). C'est le cas de la base NoSQL MongoDB.

Théorème de CAP



AP à contrario s'intéresse à fournir un temps de réponse rapide tout en distribuant les données et les réplicas. De fait, les mises à jour sont asynchrones sur le réseau, et la donnée est "Eventually Consistent" (L1 voit la version v2, tandis que L2 voit la version v1). C'est le cas de Cassandra.

Types des bases de données NOSQL

Types des bases de données NOSQL

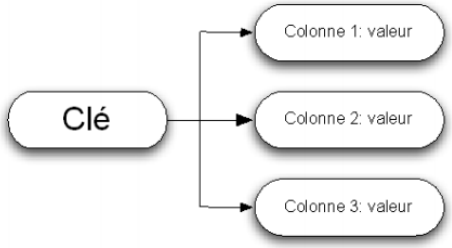
- 4 types des bases de données NOSQL:
  - Clef/valeur



redis

riak

  - Orientées colonnes

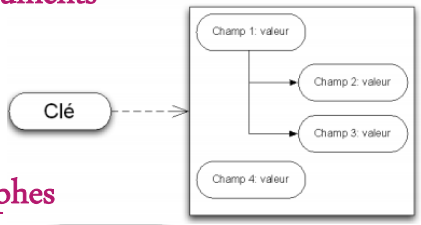


Cassandra

HBASE

Types des bases de données NOSQL

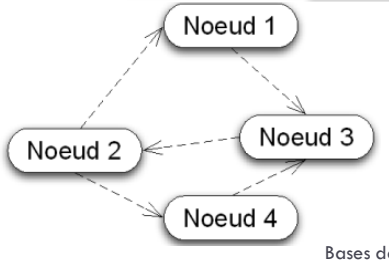
- 4 types des bases de données NOSQL:
  - Orientées documents



mongoDB

CouchDB
relax

  - Orientées graphes

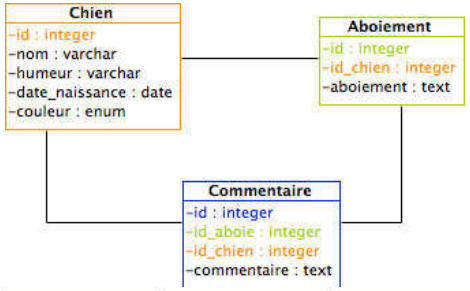


Neo4j
the graph database

HyperGraphDB

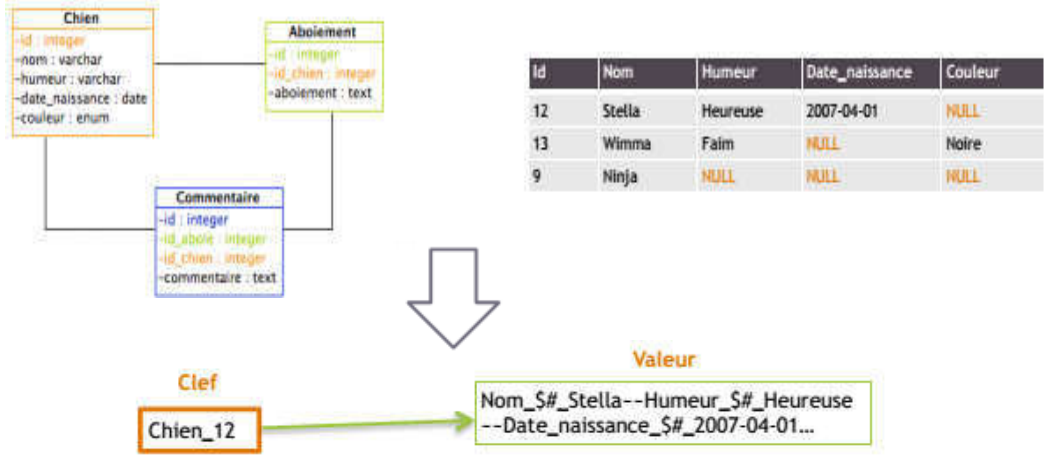
Types des bases de données NOSQL

- Exemple en relationnel:
  - NULL?



Id	Nom	Humeur	Date_naissance	Couleur
12	Stella	Heureuse	2007-04-01	NULL
13	Wimma	Faim	NULL	Noire
9	Ninja	NULL	NULL	NULL

1. Les basas de données Clef-valeur (Key/Value Store)



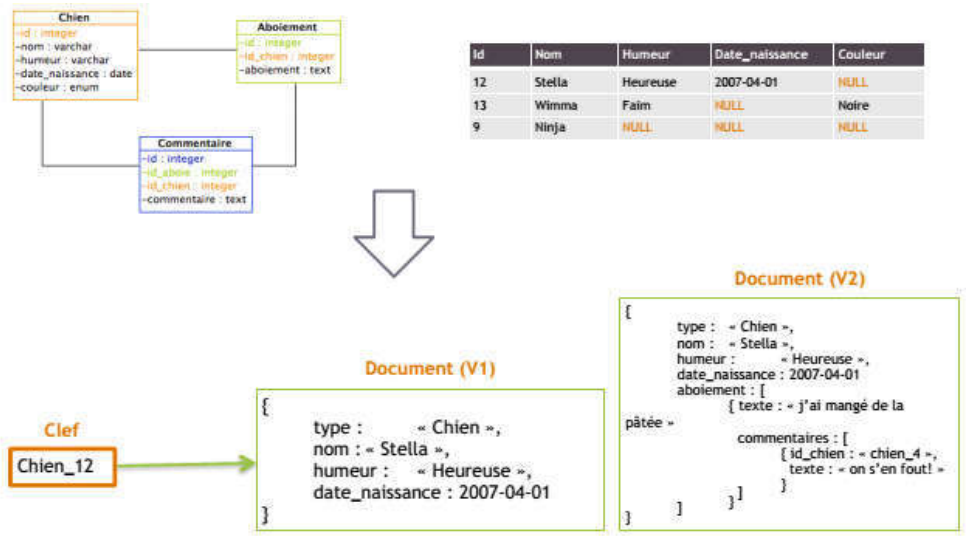
1. Les bases de données Clef-valeur (Key/Value Store)

- L'un des types les plus simples, sorte de Hashmap distribuée
- Conçu pour sauvegarder les données sans définir de schéma.
- Toutes les données sont sous forme de clef/valeur
  - La valeur peut être une chaîne de caractères, un objet sérialisé, blob…
  - La donnée est opaque au système: il n'est pas possible d'y accéder sans passer par la clef
- Absence de typage a un impact sur le requêtage: toute l'intelligence portée avant par les requêtes devra être portée par l'applicatif qui interroge la BD.
- Communications se résumant surtout aux opérations PUT, GET et DELETE.
- Objectif : fournir un accès rapide aux informations, conserver la session d'un site web…
- Exemple : DynamoDB (Amazon), Azure Table Storage (ATS), Redis, BerkeleyDB, Voldemort (LinkedIn).

1. Les basas de données Clef-valeur (Key/Value Store)

- Avantages:
  - Très rapide
  - Passage à l'échelle
  - Modèle simple
  - Distribution horizontale très facile
- Inconvénients:
  - Interrogation seulement sur clé
  - Modèle de données trop pauvre pour les données complexes
  - Déporte une partie de la complexité au niveau de l'application

2. Les bases de données orientées Documents (Document Database)



2. Les bases de données orientées Documents (Document Database)

- Étendent le paradigme clef/valeur, avec des « documents » plus complexes à la place des données simples, et une clef unique pour chacun d'eux.
- Documents de type JSON ou XML.
- Chaque document est un objet, contient un ou plusieurs champs, et chaque champs contient une valeur typée (string, date, binary ou array)
- Permettent de stocker, extraire et gérer les informations orientées documents (données semi structurées)
- Avantage : pouvoir récupérer, via une seule clef, un ensemble d'informations structurées de manière hiérarchique
  - Dans les bases relationnelles, cela impliquerait plusieurs jointures
- Exemples : MongoDB (SourceForge), CouchDB (Apache), RavenDB (destiné aux plateformes .Net/ Windows).

2. Les bases de données orientées Documents (Document Database)

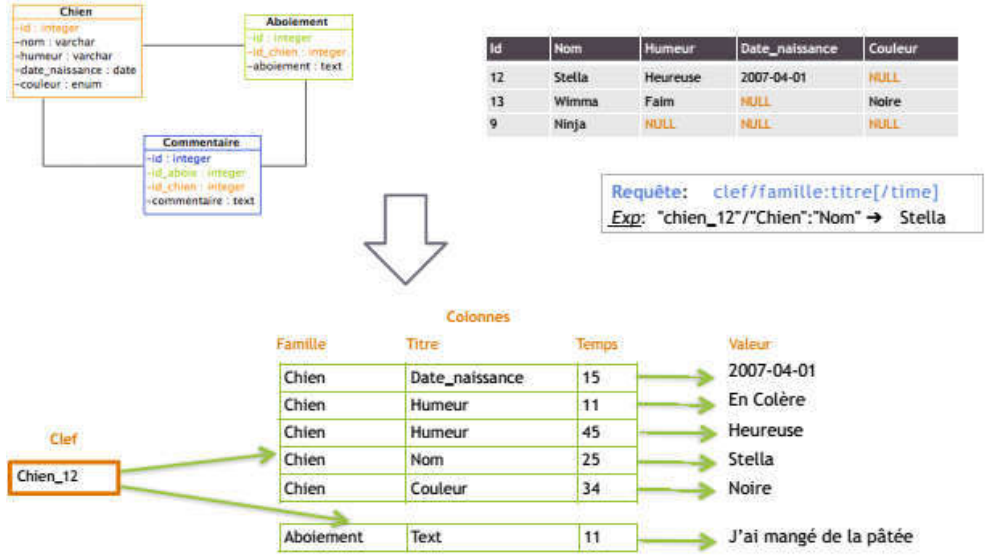
Avantages:

- modèle de données simple mais puissant
- bon passage à l'échelle
- Forte expressivité pour les requêtes (sur des structures imbriquées)

Inconvénients:

- inadapté pour les données interconnectées
- peut-être lent pour les requêtes avec MapReduce

3. Les bases de données orientées Colonnes (Column Store)



3.Les bases de données orientées Colonnes (Column Store)

ColumnFamily		
Key	**Value**	
AddressBook	SuperColumns	
	Key	**Value**
	person1	Column
		Name / **Value**
		firstName / John
		lastName / Calagan
	person2	Column
		Name / **Value**
		firstName / George
		lastName / Truffe

3. Les bases de données orientées Colonnes (Column Store)

- Évolution de la BD clef/valeur
- Ressemble aux SGBDR, mais avec un nombre de colonnes dynamique, différent d'un enregistrement à un autre (pas de colonnes portant les valeurs NULL)
- Offrent de très hautes performances et une architecture hautement évolutive
- Exemples: Hbase (Hadoop), Cassandra (Facebook, Twitter), BigTable (Google)

	A	B	C	D	E
1	Foo	Bar	Hello		
2		Tom			
3			Java	Scala	Cobol

1	A Foo	B Bar	C Hello		
2	B Tom				
3	C Java	D Scala	E Cobol		

3.Les bases de données orientées Colonnes (Column Store)

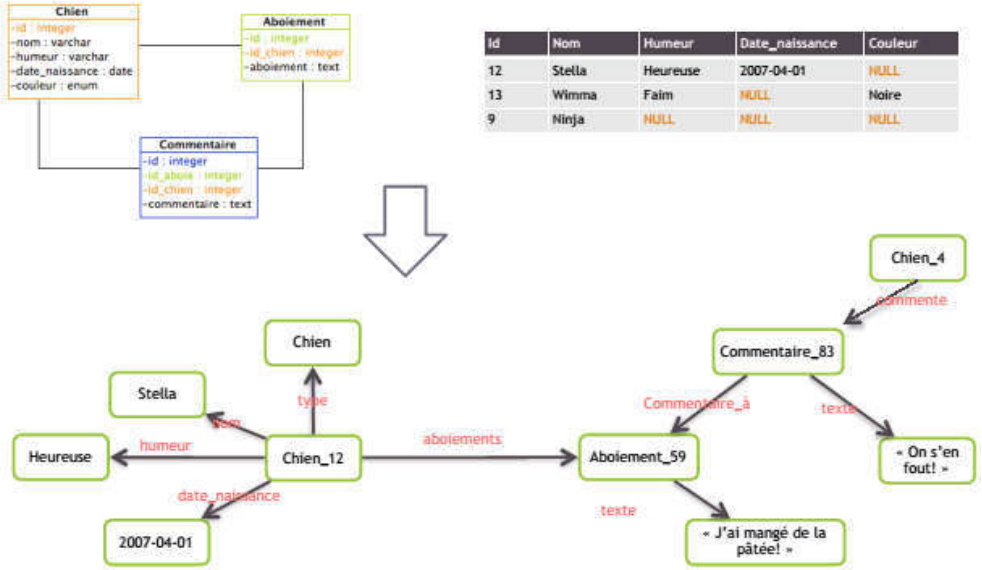
Avantages:

- modèle de données riche
- scalabilité horizontale et utilisation de MapReduce
- résultats des requêtes en temps réel

Inconvénients:

- difficile à utiliser pour les données interconnectés (distance, trajectoire)
- maintenance lors de l'ajout/suppression de colonnes
- ne pas utiliser pour les requêtes non temps réel et inconnues

4.Les bases de données orientée graphes (Graph Database)



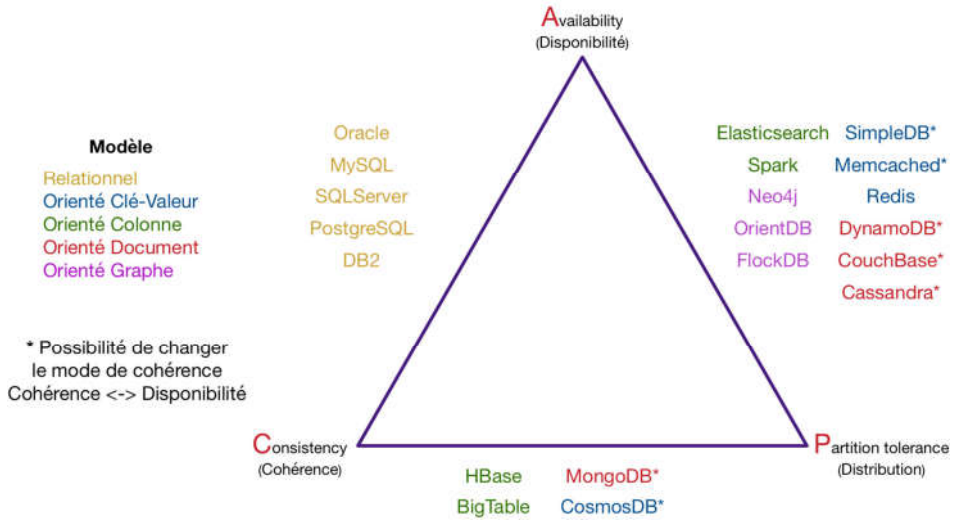
4.Les bases de données orientées graphes (Graph Database)

- Basées sur les théories des graphes.
- S'appuie sur les notions de nœuds, de relations et des propriétés qui leur sont rattachées.
- Conçues pour les données dont les relations sont représentées comme graphes, et ayant des éléments interconnectés, avec un nombre indéterminé de relations entre elles.
- Adapté aux traitements des données des réseaux sociaux.
- Exemples : Neo4J et InfiniteGraph, OrientDB.

4.Les bases de données orientée graphes (Graph Database)

- Avantages:
  - modèle de données puissant
  - rapide pour les données liées
  - modèles d'interrogation performants : Cypher, SPARQL,GraphQL
  - Algorithmes de la théorie des graphes, par exemple le plus court chemin
- Inconvénients:
  - fragmentation, possible seulement pour certains domaines
  - concept relativement récent
  - Doit traverser le graphe entier avant d'obtenir une réponse définitive

Classification des bases de données selon CAP



Top NOSQL Databases

Rank Jan 2020	Rank Dec 2019	Rank Jan 2019	DBMS	Database Model	Score Jan 2020	Score Dec 2019	Score Jan 2019
1.	1.	1.	Oracle	Relational, Multi-model	1346.68	+0.29	+77.85
2.	2.	2.	MySQL	Relational, Multi-model	1274.65	-1.01	+120.39
3.	3.	3.	Microsoft SQL Server	Relational, Multi-model	1098.55	+2.35	+58.29
4.	4.	4.	PostgreSQL	Relational, Multi-model	507.19	+3.82	+41.08
5.	5.	5.	MongoDB	Document, Multi-model	426.97	+5.85	+39.78
6.	6.	6.	IBM Db2	Relational, Multi-model	168.70	-2.65	-11.15
7.	7.	↑ 8.	Elasticsearch	Search engine, Multi-model	151.44	+1.19	+8.00
8.	8.	↓ 7.	Redis	Key-value, Multi-model	148.75	+2.51	-0.27
9.	9.	9.	Microsoft Access	Relational	128.58	-0.89	-13.04
10.	↑ 11.	10.	SQLite	Relational	122.14	+1.78	-4.66
11.	↓ 10.	11.	Cassandra	Wide column	120.66	-0.04	-2.32
12.	12.	12.	Splunk	Search engine	88.67	-1.85	+7.25
13.	13.	13.	MariaDB	Relational, Multi-model	87.45	+0.66	+8.63
14.	14.	↑ 15.	Hive	Relational	84.24	-1.81	+14.33
15.	15.	↓ 14.	Teradata	Relational, Multi-model	78.29	-0.21	+2.10
16.	16.	↑ 20.	Amazon DynamoDB	Multi-model	62.02	+0.39	+6.93
17.	17.	↓ 16.	Solr	Search engine	56.57	-0.65	-4.92

- https://db-engines.com/en/ranking

Fin